Відокремлений структурний підрозділ

«Фаховий коледж інформаційних технологій»

Національного університету «Львівська політехніка»



лабораторної роботи

з дисципліни « WEB-програмування »

Виконав:

Студент групи КН-43

Гарбіч Богдан Назарович

Перевірив:

викладач

Роман Скалоцький

Львів 2026

ЗВІТ

з лабораторної роботи №6

1. Мета роботи

Навчитися відображати дані сенсорів на існуючій сторінці «Акустична варта» та фільтрувати їх за районом і статусом. Опанувати роботу з маніпуляціями DOM (пошук елементів, динамічне створення карток, очищення та оновлення контейнерів) та подійну модель JavaScript (обробка події `change` на елементах `select` і `checkbox`).

2. Постановка задачі

1. Додати до існуючої сторінки «Акустична варта» нову секцію «Активні сенсори» з елементами фільтрації: `select` для вибору району та `checkbox` для відбору тільки критичних сенсорів.
2. Підготувати масив даних про сенсори з урахуванням статусів (`normal`, `warning`, `critical`).
3. Реалізувати динамічне створення карток сенсорів за допомогою DOM API та додавання їх у контейнер `.sensor-list`.
4. Реалізувати стилізацію статусів сенсорів відповідно до вимог та дизайну сторінки.
5. Реалізувати функцію фільтрації `applyFilters()`, яка комбінує фільтрацію за районом та за статусом.
6. Налаштувати обробку подій `change` для автоматичного оновлення списку карток при зміні параметрів фільтрації.

3. Структура HTML-секції та CSS-стилів

3.1. HTML-структура секції (`index.html`)

```
<section class="active-sensors-section">
    <div class="container">
        <h2>Активні сенсори</h2>
        <p class="section-desc">Секція показує активні сенсори та
дає змогу відфільтрувати їх за районом і статусом.</p>

        <div class="sensor-filters-card">
            <div class="filter-group">
                <label for="district-filter">Район</label>
                <select id="district-filter">
                    <option value="all">Усі райони</option>
```

```
                    <option value="Галицький">Галицький</option>
                    <option
value="Франківський">Франківський</option>
                    <option
value="Шевченківський">Шевченківський</option>
                    <option value="Сихівський">Сихівський</option>
                    <option
value="Личаківський">Личаківський</option>
                    <option value="Залізничний">Залізничний</option>
                </select>
            </div>

            <div class="filter-checkbox-group">
                <label class="checkbox-label">
                    <input type="checkbox" id="critical-filter">
                    <span>Тільки критичні</span>
                </label>
            </div>
        </div>

        <p class="filter-hint">Фільтри застосовуються автоматично
після зміни значення.</p>

        <div class="sensor-list"></div>
    </div>
</section>
```

3.2. Стилізація статусів (`style.css`)

```
/* Оформлення карток та статусних рамників */
.sensor-card {
    background: #ffffff;
    border: 1px solid #e2e8f0;
    border-radius: 12px;
    padding: 20px;
    display: flex;
    flex-direction: column;
    gap: 12px;
}

.sensor-card--normal { border-left: 4px solid #057a55; }
```

```
.sensor-card--warning { border-left: 4px solid #d97706; }
.sensor-card--critical { border-left: 4px solid #e02424; }

.sensor-status-badge {
    font-size: 12px;
    font-weight: 600;
    padding: 4px 10px;
    border-radius: 20px;
}
.sensor-card--normal .sensor-status-badge { background-color:
#def7ec; color: #03543f; }
.sensor-card--warning .sensor-status-badge { background-color:
#fef3c7; color: #92400e; }
.sensor-card--critical .sensor-status-badge { background-color:
#fde8e8; color: #9b1c1c; }
```

4. Програмна реалізація (`js/app.js`)

```
import { NoiseSensor } from './models/NoiseSensor.js';
import { MonitoringSystem } from './services/MonitoringSystem.js';

// 1. Початковий масив сенсорів
const initialSensorsData = [
    { id: "NS-01", district: "Галицький", location: "просп. Свободи,
14", noiseLevel: 52 },
    { id: "NS-02", district: "Франківський", location: "вул.
Наукова, 8", noiseLevel: 68 },
    { id: "NS-03", district: "Шевченківський", location: "просп.
Чорновола, 21", noiseLevel: 83 },
    { id: "NS-04", district: "Сихівський", location: "вул. Зелена,
102", noiseLevel: 49 },
    { id: "NS-05", district: "Личаківський", location: "вул.
Пасічна, 30", noiseLevel: 71 },
    { id: "NS-06", district: "Залізничний", location: "вул.
Городоцька, 155", noiseLevel: 86 }
];

const monitoringSystem = new MonitoringSystem();
initialSensorsData.forEach(data => {
    monitoringSystem.addSensor(NoiseSensor.fromObject(data));
});
```

```
// 2. Пошук DOM-елементів
const sensorList = document.querySelector(".sensor-list");
const districtFilter = document.querySelector("#district-filter");
const criticalFilter = document.querySelector("#critical-filter");

// 3. Створення картки сенсора
function createSensorCard(sensor) {
    const status = sensor.getStatus();

    const card = document.createElement("article");
    card.classList.add("sensor-card", `sensor-card--${status}`);

    const header = document.createElement("div");
    header.classList.add("sensor-card-header");

    const title = document.createElement("h3");
    title.textContent = sensor.id;

    const statusBadge = document.createElement("span");
    statusBadge.classList.add("sensor-status-badge");
    statusBadge.textContent = status;

    header.append(title, statusBadge);

    const district = document.createElement("p");
    district.classList.add("sensor-info-item");
    district.innerHTML = `📍 Район: <span class="label-
val">${sensor.district} район</span>`;

    const location = document.createElement("p");
    location.classList.add("sensor-info-item");
    location.innerHTML = `🏢 Локація: <span class="label-
val">${sensor.location}</span>`;

    const level = document.createElement("p");
    level.classList.add("sensor-info-item", "sensor-noise-level");
    level.innerHTML = `📊 Рівень шуму: <span class="label-
val">${sensor.noiseLevel} дБ</span>`;

    card.append(header, district, location, level);
    return card;
```

```
}

// 4. Функція відображення сенсорів
function renderSensors(sensorArray) {
    if (!sensorList) return;
    sensorList.replaceChildren(); // Очищення контейнера

    sensorArray.forEach((sensor) => {
        const card = createSensorCard(sensor);
        sensorList.append(card);
    });
}

// 5. Фільтрація сенсорів
function applyFilters() {
    const selectedDistrict = districtFilter ? districtFilter.value :
"all";
    const criticalOnly = criticalFilter ? criticalFilter.checked :
false;

    const filteredSensors = monitoringSystem.sensors.filter((sensor)
=> {
        const matchesDistrict = (selectedDistrict === "all" ||
sensor.district === selectedDistrict);
        const matchesStatus = (!criticalOnly || sensor.getStatus()
=== "critical");

        return matchesDistrict && matchesStatus;
    });

    renderSensors(filteredSensors);
}

// 6. Обробка зміни фільтрів
if (districtFilter) districtFilter.addEventListener("change",
applyFilters);
if (criticalFilter) criticalFilter.addEventListener("change",
applyFilters);

// 7. Початкове виведення
document.addEventListener('DOMContentLoaded', () => {
    renderSensors(monitoringSystem.sensors);
});
```

5. Результати виконання та перевірки

1. **Початковий стан:** Під час завантаження сторінки користувачу відображаються картки усіх 6 сенсорів із відповідними кольоровими акцентами (зелений — `normal`, жовтий — `warning`, червоний — `critical`).
2. **Фільтрація за районом:** Вибір конкретного району зі списку `select` (наприклад, "Галицький") виводить лише картки сенсорів цього району. Вибір "Усі райони" повертає вивід усіх об'єктів.
3. **Фільтрація за статусом:** При відмітці чекбокса «Тільки критичні» відображаються тільки сенсори із рівнем шуму $> 70\text{ дБ}$ (`status === "critical"`).
4. **Комбінована фільтрація:** Фільтри працюють одночасно. Якщо в обраному районі немає критичних сенсорів, контейнер `.sensor-list` залишається порожнім.

6. Відповіді на контрольні запитання

1. **Для чого використовується `document.querySelector()`?**

Для пошуку та отримання посилання на перший HTML-елемент у документації, що відповідає вказаному CSS-селектору (наприклад, `.sensor-list` або `#district-filter`).

2. **Що робить `document.createElement()`?**

Створює новий HTML-елемент у пам'яті (наприклад, `article`, `h3`, `p`), який надалі можна наповнити вмістом та додати у DOM-дерево.

3. **Для чого потрібна властивість `textContent`?**

Для безпечного встановлення або отримання текстового вмісту елемента без обробки HTML-тегів, що захищає від XSS-атак.

4. **Навіщо перед повторним відображенням очищувати `.sensor-list`?**

Щоб видалити попередній набір карток і уникнути дублювання елементів під час повторного виклику `renderSensors()`. У коді для цього використано `replaceChildren()`.

5. **Що повертає метод `filter()`?**

Новий масив, що містить тільки ті елементи вихідного масиву, які відповідають умові (для яких функція-предикат повернула `true`).

6. **Як отримати вибране значення з `select`?**

Через властивість `.value` відповідного DOM-елемента `select` (наприклад, `districtFilter.value`).

7. **Як перевірити, чи `checkbox` увімкнений?**

За допомогою булевої властивості `.checked` (наприклад, `criticalFilter.checked`).

8. **Чому після зміни фільтра потрібно знову викликати `renderSensors()`?**

Тому що зміна фільтра створює новий відфільтрований масив даних у пам'яті, і щоб користувач побачив оновлені дані на екрані, потрібно заново згенерувати DOM-елементи.

7. Висновки

Під час виконання лабораторної роботи було опановано практичні навички маніпулювання DOM-структурою за допомогою JavaScript. Реалізовано динамічне створення та виведення карток сенсорів, додано стилізацію статусів та налаштовано автоматичну комбіновану фільтрацію за допомогою методів масивів та обробників подій `change`.